

- Vision Guided Arm -

```

Camera sees cube
  ↓
Vision detects cube pixels (u, v)
  ↓
Estimate cube position in camera coordinates
  ↓
Transform camera coordinates → robot-base coordinates
  ↓
Check arm reachability
  ↓
IK calculates desired joint angles
  ↓
PD/PID produces actuator commands
  ↓
Arm moves
  ↓
Camera observes the result
  ↻
    
```

> how it differs from prev.

section 8:

pan/tilt only needed to
centre target

pixel error → camera rotation

⇒ arm must place its end-
effector at the cube

pixels → physical position → robot frame
joints ← IK ←

A cube at pixel (400, 200) does not tell IK how many millimetres away it is. That pixel corresponds to a 3D ray extending from the camera.

to find physical point on that ray

pixel

→ camera ray

→ intersection w/ table plane

→ cube position in 3D

eye-to-hand

fixed overhead camera

↓

sees cube and
planar arm

$P_{camera} \rightarrow P_{robot} = T_{robot \leftarrow camera} P_{camera}$

Then

$$(\theta_1^*, \theta_2^*) = \text{IK}(P_{\text{robot}})$$

↳ enters the inner PD/PID controller

1. Why can't we send cube pixels (u, v) directly into arm IK?
2. What extra assumption lets one image pixel become a unique physical position in our first experiment?
3. Which transform converts the estimated cube position into the arm's coordinate frame?
4. What does IK output?
5. Would reading the cube's exact MuJoCo body coordinates count as vision-guided control?

1. it doesn't tell how many mm away it is to IK;
it says the 3D ray extending from the cam.

2. the cube will sit on a known flat plane

$$3. P_{\text{robot}} = T_{\text{robot} \leftarrow \text{camera}} P_{\text{camera}}$$

4. desired joints for controllers

5. no it'd be ground truth

For an OpenCV-style camera frame:

$$u = f_x \frac{X_c}{Z_c} + c_x$$

$$v = f_y \frac{Y_c}{Z_c} + c_y$$

Rearranging:

$$X_c = (u - c_x) \frac{Z_c}{f_x}$$

$$Y_c = (v - c_y) \frac{Z_c}{f_y}$$

Without Z_c , we only get a ray:

$$r_c = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}$$

The known plane supplies Z_c in our simple overhead, perpendicular-camera setup.

EXAMPLE

$$f_x = f_y = 500 \text{ px}$$

$$(c_x, c_y) = (320, 240)$$

therefore

$$u = 370$$

$$v = 215$$

$$z_c = 1000 \text{ mm}$$

$$x_c = (370 - 320) \frac{1000}{500} = 100 \text{ mm}$$

$$y_c = (215 - 240) \frac{1000}{500} = -50 \text{ mm}$$

$$\Rightarrow p_c = \begin{bmatrix} 100 \\ -50 \\ 1000 \end{bmatrix} \text{ mm}$$

expressed in
cam. frame

> camera coords $\rightarrow$ robot coords

Assume:

- Camera is 500 mm directly above the robot origin.
- Camera looks straight downward.
- Camera x aligns with robot x .
- Camera y points opposite robot y .
- Camera z points downward, opposite robot z .

$$R_{\text{robot} \leftarrow \text{camera}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} ; t_{\text{robot} \leftarrow \text{camera}} = \begin{bmatrix} 0 \\ 0 \\ 500 \end{bmatrix}$$

$$\Rightarrow T_{\text{robot} \leftarrow \text{camera}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 500 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$p_r = T_{\text{robot} \leftarrow \text{cam}} \cdot p_c = \begin{bmatrix} 100 \\ -50 \\ 0 \\ 1 \end{bmatrix}$$

$\rightarrow$ 100 right (+x)
50 up (-y)
lies on robot plane

> given $p_c = \begin{bmatrix} 60 \\ -20 \end{bmatrix}$

$$\begin{bmatrix} 50 \\ 0 \\ 1 \end{bmatrix}$$

$$\hookrightarrow p_r = \begin{bmatrix} 60 \\ 20 \\ 0 \\ 1 \end{bmatrix}$$

$\rightarrow$ x and y passed into planar arm IK

for $L_1 = 100\text{mm}$, $L_2 = 50\text{mm}$

$$50 < 20\sqrt{10} < 150 \rightarrow \text{reachable}$$

$$\Rightarrow (x_r, y_r) = (60, 20)$$

$$\cos \theta_2 = \frac{60^2 + 20^2 + 100^2 - 50^2}{2 \cdot 100 \cdot 50} = -0.85$$

$$\theta_2 = \pm \arccos(-0.85)$$

Solution 1: $\theta_1 = -6.17^\circ$; $\theta_2 = 148.21^\circ$

Solution 2: $\theta_1 = 43.04^\circ$; $\theta_2 = -148.21^\circ$

Both place the end-effector at (60, 20), but their elbow configurations differ.

This creates an important integration problem: noisy camera measurements must not make IK alternate randomly between the two branches. Otherwise, nearly identical cube positions could produce enormous joint-target jumps.

A common selection policy is:

From all reachable, joint-limit-safe IK solutions, choose the one closest to the current joint configuration.

1. Why are there two valid joint solutions for one cube position?
2. What could happen if the program chooses a different IK branch on every camera frame?
3. Why might "closest to current qpos" be a better selection rule than always choosing Solution 1?
4. After choosing one solution, does it go directly into ctrl, or does it first become something else?

$\perp$ reachable points can be reached w/ 2 elbow config.

2. it can produce joint-target jumps

3. "Closest to current qpos" provides:
- Less joint movement.
 - Smoother frame-to-frame targets.
 - Lower chance of sudden elbow flipping.
 - Usually lower control effort.
 - Better continuity with the robot's present posture.

4. compare w/ qpos/qvel → ctrllers (PD/PID)

> Virtual camera Intrinsics

simulated overhead cam: 60° ; 640×480

$$f_y = \frac{H/2}{\tan(f_{\text{fov}}/2)} \quad \rightarrow \text{for real Mac camera we'd use calibrated } K \text{ from section 1}$$
$$= \frac{240}{\tan 30^\circ} \approx 415.7 \text{ px} = f_x = f_y$$

$$(c_x, c_y) = (320, 240)$$

$$Z_c = 500 \text{ mm}$$

$$\hookrightarrow \frac{500}{415.7} \approx 1.20 \text{ mm/pixel}$$

→ inspect overhead camera

```
Image shape: (480, 640, 3)
Vertical FOV: 60.0
Focal length: 415.6921938165307
Camera matrix K:
[[415.69219382  0.  320.  0.]
 [  0.  415.69219382 240.  0.]
 [  0.  0.  1.  0.]]
```

→ estimate target position

```

Detected: True
Target area: 361
Target pixel centre: (411.994459833795, 193.05263157894737)

Camera ray:
[ 0.22130428 -0.11293782 1.      ]

Estimated camera position (m):
[ 0.09958692 -0.05082202 0.45    ]

Trobot←camera:
[[ 1.  0.  0.  0. ]
 [ 0. -1. 0.  0. ]
 [ 0.  0. -1. 0.5]
 [ 0.  0.  0.  1. ]]

Estimated robot position (m):
[0.09958692 0.05082202 0.05      ]

Ground-truth target centre (m):
[0.1 0.05 0.04]

Planar estimation error: 0.9199695092041754 mm

```

1. Why did camera $Y_c = -0.0508$ become robot $Y_r = +0.0508$?
2. Why is estimated $z = 0.05$ different from the body-centre ground truth $z = 0.04$?
3. Which values should planar IK receive?
4. Should the later controller receive the estimated robot position or the printed ground-truth position?
5. What does the 0.92 mm error measure?

1. cuz of the Trobot←cam contains -1 on y-axis
2. 0.05mm → visible cube top ; 0.04mm cube body centre
3. x and y
4. estimated robot pos.
5. ground truth vs. estimated robot position (euclidian difference)

→ solve visual IK

```

Detected: True
Target area: 361
Target pixel centre: (411.994459833795, 193.05263157894737)

IK solutions:
Solution 1: [ 0.47194812 89.99751989] degrees
Solution 2: [ 53.60105841 -89.99751989] degrees

Current joints: [0. 0.] degrees
Selected solution: [ 0.47194812 89.99751989] degrees
FK verification position: [0.09958692 0.05082202] m
IK verification error: 6.938893903907228e-15 mm

Camera ray:
[ 0.22130428 -0.11293782 1.      ]

```

↪ consider

```

Perception error:
Pest versus Ptrue

IK error: ≈ 0
FK(q*) versus Pest

Control error:
qpos versus q*

```

$$e = q^* - q_{pos}$$

> perception error:

true target → estimated target

$$\text{error} = 0.82 \text{ mm}$$

IK:

estimated

Estimated camera position (m):
[0.09958692 -0.05082202 0.45

P_{est}

Trobot<-camera:
[[1. 0. 0. 0.]
[0. -1. 0. 0.]
[0. 0. -1. 0.5]
[0. 0. 0. 1.]]

→ IK receives

Estimated robot position (m):
[0.09958692 0.05082202 0.05]

Ground-truth target centre (m):
[0.1 0.05 0.04]

Planar estimation error: 0.9199695092041754 mm

→ perception error

estimated target → FK(selected joints)

error ≈ 0.

→ end-to-end error chain;

camera estimation error
+ IK numerical/approximation error
+ joint-control tracking error
→ actual end-effector error

True cube

↓ perception error

Estimated cube

↓ IK produces q^*

Desired joints

↓ control error

Actual qpos

↓ FK

Actual end-effector position

1. Why is θ_1 approximately 0.47° instead of exactly 0° ?
2. Why can IK error be almost zero while perception error is 0.92 mm?
3. If joint control were perfect, approximately how far from the true cube position might the end-effector still finish?
4. If actual qpos does not reach the desired joints, which error source is responsible?

1. because vision est. (0.0995, 0.0508) instead of ideal (0.100, 0.0500)

2. comparing FK(q^) against P_{est} ; IK produced the est target perfectly but estimate differs from P_{true} by 0.92 mm*

3. 0.92 mm from the cube

4. joint control tracking error

→ arm moving to visual target

Detected: True
Target area: 361
Target pixel centre: (411.994459833795, 193.05263157894737)

IK solutions:
Solution 1: [0.47194812 89.99751989] degrees
Solution 2: [53.60105841 -89.99751989] degrees

Current joints: [0. 0.] degrees
Selected solution: [0.47194812 89.99751989] degrees
FK verification position: [0.09958692 0.05082202] m
IK verification error: 6.938893903907228e-15 mm

Camera ray:

> elbow joint rotated the 2nd link

→ shoulder target = 0.47°

→ elbow target = 90°

```

[ 0.22130428 -0.11293782 1. ]

Estimated camera position (m):
[ 0.09958692 -0.05082202 0.45 ]

Trotot<-camera:
[[ 1. 0. 0. ]
 [ 0. -1. 0. ]
 [ 0. 0. -1. ]
 [ 0. 0. 0. 1. ]]

Estimated robot position (m):
[0.09958692 0.05082202 0.05 ]

Ground-truth target centre (m):
[0.1 0.05 0.04]

Planar estimation error: 0.9199695092041754 mm

time | actual joints (deg) | joint error (deg) | ctrl | task error (mm)
0.00 | [0. 0.] | [0.47194812 89.99751989] | [0.04118524 1.] | 71.585
0.25 | [0.2483426 35.45012869] | [0.22360553 54.5473912] | [9.33194983e-04 1.00000000e+00] | 46.178
0.50 | [0.58400605 70.41313365] | [-0.11205793 19.58438623] | [-0.02286955 0.7489647] | 16.879
0.75 | [0.59353735 85.15104845] | [-0.12158923 4.84647144] | [-0.00662766 0.18541018] | 4.119
1.00 | [0.51166844 88.79771486] | [-0.03972031 1.19980503] | [-0.00176184 0.04590798] | 1.014
1.25 | [0.48315006 89.70046297] | [-0.01120193 0.29705692] | [-0.00046851 0.01136689] | 0.250
1.50 | [0.47496514 89.92396878] | [-0.00301701 0.0735511] | [-0.000123 0.00281453] | 0.062
1.75 | [0.47274262 89.97930809] | [-0.0007945 0.0182118] | [-3.19074254e-05 6.96917949e-04] | 0.015
2.00 | [0.47215439 89.99301039] | [-0.00020627 0.00450949] | [-8.19778059e-06 1.72570141e-04] | 0.004
2.25 | [0.47200113 89.99640325] | [-5.30110460e-05 1.11663576e-03] | [-2.09052774e-06 4.27323713e-05] | 0.001
2.50 | [0.47196164 89.99724338] | [-1.35204938e-05 2.76504844e-04] | [-5.30012978e-07 1.05816741e-05] | 0.000
2.75 | [0.47195155 89.99745142] | [-3.42820531e-06 6.84699532e-05] | [-1.33761246e-07 2.62033455e-06] | 0.000
3.00 | [0.47194899 89.99750293] | [-8.65252951e-07 1.69551764e-05] | [-3.36357154e-08 6.48878225e-07] | 0.000
3.25 | [0.47194834 89.99751569] | [-2.17589921e-07 4.19864060e-06] | [-8.43366218e-09 1.60684143e-07] | 0.000
3.50 | [0.47194818 89.99751885] | [-5.45600106e-08 1.03972495e-06] | [-2.10972193e-09 3.97910698e-08] | 0.000
3.75 | [0.47194814 89.99751963] | [-1.36489625e-08 2.57472542e-07] | [-5.26772543e-10 9.85372361e-09] | 0.000
4.00 | [0.47194813 89.99751982] | [-3.40808590e-09 6.37596128e-08] | [-1.31330305e-10 2.44015272e-09] | 0.000
4.25 | [0.47194812 89.99751987] | [-8.49694625e-10 1.57893039e-08] | [-3.27019866e-11 6.04276765e-10] | 0.000
4.50 | [0.47194812 89.99751988] | [-2.11583120e-10 3.91002126e-09] | [-8.13489601e-12 1.49641988e-10] | 0.000
4.75 | [0.47194812 89.99751989] | [-5.26335085e-11 9.68300789e-10] | [-2.02197595e-12 3.70580747e-11] | 0.000

Final joints: [ 0.47194812 89.99751989] degrees
Final end-effector: [0.09958692 0.05082202] m
Error to estimated target: 1.9912678843823082e-10 mm
Error to true target: 0.9199695091372913 mm

```

IK error = 0

Control error = 0

Perception error = 0.92 mm

1. Why did most visible movement come from the elbow rather than the shoulder?
2. Which output shows actuator saturation at the beginning?
3. Why is error to the estimated target nearly zero while error to the true cube remains 0.92 mm?
4. Did vision run continuously during this movement, or only once at the beginning?
5. Which loop remained continuously closed during the movement?

1. becu2 degrees are very distort ; one is 0.47 and the other is 80
2. ctrl
3. it comes from the perception error at the beginning
4. once for perception only
5. inner joint-control loop →

```

desired joints
↓
compare with qpos/qvel
↓
PD calculates ctrl
↓
MuJoCo moves arm
↓
new qpos/qvel
↺

```

1. In a continuous vision-guided loop, which value should be recalculated whenever a new camera frame arrives?
2. If vision runs around 30 Hz and joint control around 500 Hz, which is the inner/faster loop?
3. Between camera frames, should the controller discard the desired joints or keep using the latest ones?
4. If the cube moves after the first frame in our current script, will the arm automatically follow it?

2. outer loop (pixels → cam pos → robot pos. → IK → new desi. joints)
2. Zero-order hold:

(Joint control is faster)

keep using the most recent until the next can update

3. keep

4. it will go to the old pos.

→ continuous visual arm

time	actual joints (deg)	joint error (deg)	ctrl	task error (mm)
0.00	[0. 0.]	[0.47194812 89.99751989]	[0.04118524 1.]	71.585
0.25	[0.63080274 35.41369059]	[1.14724472 50.76916809]	[0.04157995 1.]	44.783
0.50	[2.54796639 68.20735223]	[1.20694983 14.80913837]	[0.03069765 0.5662104]	14.598
0.75	[4.59952805 77.53363053]	[1.49722492 -1.18341072]	[0.08128317 -0.14674525]	2.688
1.00	[6.3284008 74.68539528]	[1.23661639 -3.39037813]	[0.04136609 -0.13034088]	2.412
1.25	[7.94556396 70.5167828]	[0.94460227 -2.38595835]	[0.0312802 -0.09184768]	1.684
1.50	[9.4134713 66.34810737]	[1.06995536 -2.74470412]	[0.03705785 -0.10648157]	1.820
1.75	[10.7010387 62.86975707]	[0.54868526 -1.77348907]	[0.01423246 -0.06868643]	1.048
2.00	[11.75084483 59.48544463]	[0.68894424 -2.80176031]	[0.02251947 -0.10781995]	1.581
2.25	[12.19108593 57.22898215]	[0.2232349 -1.47394361]	[0.00602278 -0.05556116]	0.935
2.50	[12.35782053 56.13280458]	[0.22292279 -1.44926091]	[0.00934413 -0.05546917]	0.908
2.75	[12.47390912 54.89964213]	[-0.52416725 0.28078669]	[-0.0256058 0.01011761]	1.051
3.00	[11.876211 55.53211498]	[-0.85511308 1.12823991]	[-0.0393579 0.04217744]	1.374
3.25	[10.89845625 56.81944828]	[-0.70194746 0.92858943]	[-0.02440316 0.03589573]	1.131
3.50	[9.65550885 58.64010252]	[-0.83841389 1.30970887]	[-0.02901931 0.05056748]	1.302
3.75	[8.23235597 60.50698182]	[-1.23619838 1.30351224]	[-0.0535332 0.04931373]	2.130
4.00	[6.39567777 64.03791625]	[-1.15503226 1.74634035]	[-0.041127 0.06726881]	1.857
4.25	[4.44804033 67.59213356]	[-1.34549723 2.43164124]	[-0.04735042 0.09360748]	2.192
4.50	[2.51062878 71.0683005]	[-1.34586527 1.76381641]	[-0.05241547 0.06750527]	2.263
4.75	[0.52023422 74.8482838]	[-1.27444159 2.87469883]	[-0.04664574 0.11030782]	2.329
5.00	[-1.48405837 78.84717283]	[-1.50927743 1.76956613]	[-0.07032425 0.02893639]	2.603
5.25	[-3.36704153 82.62584413]	[-1.20570694 2.31939591]	[-0.04440357 0.08894083]	2.220
5.50	[-5.08637299 86.47402799]	[-1.09358966 2.36709859]	[-0.03908384 0.09079991]	2.169
5.75	[-6.66211914 90.2454958]	[-1.05426229 2.06029963]	[-0.04857019 0.07849282]	2.056
6.00	[-7.94714032 93.63215103]	[-0.866678 2.14740996]	[-0.03448923 0.08212298]	1.953
6.25	[-8.99557052 96.83669283]	[-0.58666847 2.01036069]	[-0.02112783 0.07698194]	1.714
6.50	[-9.69327161 100.05481394]	[-0.5108031 1.65710246]	[-0.01966967 0.06339948]	1.460
6.75	[-10.094484 102.84178021]	[-0.08290527 1.22133716]	[-0.00324935 0.04672531]	1.037
7.00	[-10.07591277 105.38250754]	[0.04226189 1.06946469]	[0.00080823 0.04090036]	0.953
7.25	[-9.70683511 107.54664541]	[0.49272176 1.14668208]	[0.01169258 0.04378389]	1.422
7.50	[-9.00270214 109.29924901]	[0.91812865 0.96683422]	[0.04039223 0.03703815]	1.868
7.75	[-7.89970013 110.57253523]	[0.6125825 0.00936105]	[0.02022486 0.00034893]	1.015
8.00	[-6.49000601 111.57187065]	[0.86145568 0.58259294]	[0.02485817 0.02228435]	1.561
8.25	[-4.90970301 112.08866049]	[0.93434351 0.2838417]	[0.03201371 0.01086891]	1.574
8.50	[-3.18184236 111.99718007]	[1.43622683 -0.29301958]	[0.05517076 -0.01118383]	2.327
8.75	[-1.33195545 111.60158218]	[1.65752907 -1.20793579]	[0.08007936 -0.04615734]	2.777
9.00	[0.46024837 110.37635781]	[1.40411736 0.12164079]	[0.05363185 0.00467805]	2.345
9.25	[2.43076436 109.24724482]	[1.2638619 -1.34709025]	[0.04991888 -0.06547817]	2.230
9.50	[4.14585266 107.37278865]	[1.15469474 -1.3321868]	[0.03903934 -0.05101126]	2.065
9.75	[5.77345617 105.36711658]	[1.35283876 -1.86458774]	[0.05315084 -0.07128617]	2.510
10.00	[7.20951332 102.87951213]	[0.87443074 -1.88280884]	[0.03415034 -0.070656]	1.913
10.25	[8.42457214 100.43687078]	[0.65781821 -2.09607746]	[0.02641678 -0.08015229]	1.834
10.50	[9.4819218 98.0247366]	[0.28763166 -2.13291189]	[0.00455155 -0.08184286]	1.744
10.75	[10.20496775 95.39659459]	[0.72828024 -1.83646174]	[0.04197134 -0.06964704]	1.656
11.00	[10.67676365 93.12614636]	[0.70849158 -1.80100795]	[0.03666594 -0.06845486]	1.590
11.25	[10.93986457 90.80598858]	[-0.17582514 -1.29291478]	[-0.00608751 -0.04944437]	1.317
11.50	[10.75901977 89.15459509]	[-0.18780579 -2.14499767]	[-0.01283832 -0.0824007]	2.073
11.75	[10.57719182 87.03852038]	[-0.24534374 -1.23463373]	[-0.00935577 -0.0472524]	1.386

Final joints: [9.96531035 85.32427186] degrees
Final end-effector: [0.09388179 0.06709226] m
Error to estimated target: 1.1973835470824192 mm
Error to true target: 1.8082963011103461 mm

The output confirms genuine tracking:

- Initial task error fell from 71.6 mm to 2.7 mm in 0.75 s.
- While the cube moved, error generally stayed around 1–3 mm.
- Joint angles changed continuously and smoothly.
- Elbow remained on the positive-angle branch—no IK branch flipping.
- After the initial transient, control stayed far from saturation.
- Final error to the latest estimated target was 1.20 mm.
- Final error to the true target was 1.81 mm.

The moving target prevents error from settling permanently at zero because:

- Vision updates around 30 Hz.
- Desired joints are held between frames.
- Motors have finite response speed.
- The target keeps moving.
- Perception still has small estimation error.

→ arm lag/tracking relative to latest estimate

] different

→ complete miss relative to the real target

> now we have complete!

rendered camera
→ green-cube detection
→ pixel ray
→ robot-frame target
→ reachability
→ IK branch selection
→ desired joints
→ joint PD
→ moving arm
→ new camera frame
↻

1. What proves that the outer vision/IK setpoint changed during the run rather than staying at the initial (0.47°, 90°)?
2. Why does a continuously moving target usually retain some tracking error?

2. Why does a continuously moving target usually retain some tracking error?
3. Why is error to the true target larger than error to the estimated target here?
4. What evidence suggests IK did not switch randomly between elbow branches?
5. If we move the cube much faster, what do you expect to happen to task error and actuator commands?

1. $t=0$ vs. $t=3$; there is evidence of reconstructing desired joints

2. arm leg

3. there is additional error from P_{est} vs. P_{true}

4. elbow remained on the positive angle branch

5.
 - Larger tracking lag and task error.
 - Larger actuator commands.
 - Possible saturation.
 - Potential overshoot or oscillation.
 - Eventually, possible target loss if the system cannot keep up.

→ unreachable target test

task error
jumps to
88.4 mm

time	status	actual joints (deg)	joint error (deg)	ctrl	task error (mm)
0.00	initializing	[0. 0.]	[0.47194812 89.99751989]	[0.04118524 1.]	71.585
0.25	tracking	[0.63080274 35.41369059]	[1.14724472 50.76916809]	[0.04157995 1.]	44.783
0.50	tracking	[2.54796839 68.20735223]	[1.20694983 14.80913837]	[0.03060765 0.5662104]	14.598
0.75	tracking	[4.59952805 77.53363053]	[1.49722492 -1.18341072]	[0.08128317 -0.14674525]	2.688
1.00	tracking	[6.3284008 74.68539528]	[1.23661639 -3.39037813]	[0.04136609 -0.13034088]	2.412
1.25	tracking	[7.94556396 70.5167828]	[0.94460227 -2.38595835]	[0.0312802 -0.09184768]	1.684
1.50	tracking	[9.4134713 66.34810737]	[1.06995536 -2.74470412]	[0.03705785 -0.10648157]	1.820
1.75	tracking	[10.7010387 62.86975707]	[0.54868526 -1.77348907]	[0.01423246 -0.06868643]	1.048
2.00	tracking	[11.75084483 59.48544463]	[0.68894424 -2.80176031]	[0.02251947 -0.10781995]	1.581
2.25	tracking	[12.19108593 57.22898215]	[0.2232349 -1.47394361]	[0.00602278 -0.05556116]	0.935
2.50	tracking	[12.35782053 56.13280458]	[0.22292279 -1.44926091]	[0.00934413 -0.05546917]	0.908
2.75	tracking	[12.47390912 54.89964213]	[-0.52416725 0.28078669]	[-0.0256058 0.01011761]	1.051
3.00	tracking	[11.876211 55.53211498]	[-0.85511308 1.12823991]	[-0.0393579 0.04217744]	1.374
3.25	tracking	[10.89845625 56.81944828]	[-0.70194746 0.92858943]	[-0.02440316 0.03589573]	1.131
3.50	tracking	[9.65550885 58.64010252]	[-0.83841389 1.30970887]	[-0.02901931 0.05056748]	1.302
3.75	tracking	[8.23235597 60.50698182]	[-1.23619838 1.30351224]	[-0.0535332 0.04931373]	2.130
4.00	tracking	[6.39567777 64.03791625]	[-1.15503226 1.74634035]	[-0.041127 0.06726881]	1.857
4.25	tracking	[4.44804033 67.59213356]	[-1.34549723 2.43164124]	[-0.04735042 0.09360748]	2.192
4.50	tracking	[2.51062878 71.0683005]	[-1.34586527 1.76381641]	[-0.05241547 0.06750527]	2.263
4.75	tracking	[0.52023422 74.8482838]	[-1.27444159 2.87469883]	[-0.04664574 0.11030782]	2.329
5.00	tracking	[-1.48405837 78.84717283]	[-1.50927743 1.76956613]	[-0.07032425 0.02893639]	2.603
5.25	unreachable	[-1.8483176 79.13985795]	[5.40463525e-07 4.23737158e-08]	[2.91309800e-07 2.28394283e-08]	88.384
5.50	unreachable	[-1.84831806 79.13985791]	[8.30522341e-13 3.81666562e-14]	[2.05811279e-13 1.49861816e-14]	88.384
5.75	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[2.78274672e-19 2.18174405e-20]	88.384
6.00	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[1.96456363e-24 1.54026775e-25]	88.384
6.25	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[1.38694270e-29 1.08739828e-30]	88.384
6.50	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[9.79153862e-35 7.67681478e-36]	88.384
6.75	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[6.91263080e-40 5.41967799e-41]	88.384
7.00	unreachable	[-1.84831806 79.13985791]	[0. 0.]	[4.88017935e-45 3.82618447e-46]	88.384
7.25	tracking	[-8.01454081 101.8357679]	[-2.23055738 10.07915448]	[-0.08658674 0.38567393]	8.801
7.50	tracking	[-8.89112975 108.82861304]	[0.73005616 1.79108517]	[0.03417602 0.06858328]	2.138
7.75	tracking	[-7.89726833 110.54637205]	[0.61610506 0.06343894]	[0.02045746 0.00241799]	1.029
8.00	tracking	[-6.4903158 111.56707915]	[0.86176547 0.58738444]	[0.02486771 0.02246756]	1.564
8.25	tracking	[-4.90976594 112.08747441]	[0.93440644 0.28502778]	[0.03201568 0.01091427]	1.574
8.50	tracking	[-3.18493572 111.99781316]	[1.43932019 -0.29365267]	[0.0553076 -0.01120793]	2.332
8.75	tracking	[-1.33140196 111.60186622]	[1.63396637 -1.20146958]	[0.078549 -0.04591151]	2.739
9.00	tracking	[0.4592189 110.37666633]	[1.40514683 0.12133227]	[0.05366512 0.00466625]	2.347
9.25	tracking	[2.43054891 109.24732117]	[1.26407735 -1.3471666]	[0.04992584 -0.0654811]	2.230
9.50	tracking	[4.14580762 107.37280759]	[1.15473978 -1.33220574]	[0.03904079 -0.05101198]	2.065
9.75	tracking	[5.77344677 105.36712129]	[1.35284816 -1.86459245]	[0.05315115 -0.07128635]	2.510
10.00	tracking	[7.20951136 102.8795133]	[0.87443269 -1.88281001]	[0.0341504 -0.07065605]	1.913
10.25	tracking	[8.42457173 100.43687107]	[0.65781862 -2.09607775]	[0.0264168 -0.08015231]	1.834
10.50	tracking	[9.48192171 98.02473667]	[0.28763174 -2.13291196]	[0.00455155 -0.08184287]	1.744
10.75	tracking	[10.20496774 95.39659461]	[0.72828026 -1.83646175]	[0.04197134 -0.06964704]	1.656
11.00	tracking	[10.67676365 93.12614636]	[0.70849158 -1.80100795]	[0.03666594 -0.06845486]	1.590
11.25	tracking	[10.93986457 90.80598858]	[-0.17582514 -1.29291478]	[-0.00608751 -0.04944437]	1.317
11.50	tracking	[10.75901977 89.15459509]	[-0.18780579 -2.14499767]	[-0.01283832 -0.0824007]	2.073
11.75	tracking	[10.57719182 87.03852038]	[-0.24534374 -1.23463373]	[-0.00935577 -0.0472524]	1.386

Final joints: [9.96531035 85.32427186] degrees
 Final end-effector: [0.09388179 0.06709226] m
 Error to estimated target: 1.1973835470859828 mm
 Error to true target: 1.8082963011113569 mm

see target → estimate position → check reachability → reject unsafe cmd → actively hold current posture

1. Was the unreachable cube still detected?
2. Which stage rejected it: perception, IK, or the controller?
3. Why did the task error stay at 88.384 mm while the joint error became zero?
4. When reachability returned, why wasn't the task error immediately zero?

1. yes

2. IK reachability gate rejects it

3. "hold current pos." so joint error stayed 0, task error jumped 88.4 mm away

4. around 7.25 s; becu7 arm not teleporting immediately so it stayed 8,801

→ "not detected" case

stop pursuing → brake → hold the current pos. → wait for fresh detection
the old target

time	status	actual joints (deg)	joint error (deg)	ctrl	task error (mm)
0.00	initializing	[0. 0.]	[0.47194812 89.99751989]	[0.04118524 1.]	71.585
0.25	tracking	[0.63080274 35.41369059]	[1.14724472 50.76916809]	[0.04157995 1.]	44.783
0.50	tracking	[2.54796839 68.20735223]	[1.20694983 14.80913837]	[0.03069765 0.5662104]	14.598
0.75	tracking	[4.59952805 77.53363053]	[1.49722492 -1.18341072]	[0.08128317 -0.14674525]	2.688
1.00	tracking	[6.3284008 74.68539528]	[1.23661639 -3.39037813]	[0.04136609 -0.13034088]	2.412
1.25	tracking	[7.94556396 70.5167828]	[0.94460227 -2.38595835]	[0.0312802 -0.09184768]	1.684
1.50	tracking	[9.4134713 66.34810737]	[1.06995536 -2.74470412]	[0.03705785 -0.10648157]	1.820
1.75	tracking	[10.7010387 62.86975707]	[0.54868526 -1.77348907]	[0.01423246 -0.06868643]	1.048
2.00	tracking	[11.75084483 59.48544463]	[0.68894424 -2.80176031]	[0.02251947 -0.10781995]	1.581
2.25	tracking	[12.19108593 57.22898215]	[0.2232349 -1.47394361]	[0.00602278 -0.05556116]	0.935
2.50	tracking	[12.35782053 56.13280458]	[0.22292279 -1.44926091]	[0.00934413 -0.05546917]	0.908
2.75	tracking	[12.47390912 54.89964213]	[-0.52416725 0.28078669]	[-0.0256058 0.01011761]	1.051
3.00	tracking	[11.876211 55.53211498]	[-0.85511308 1.12823991]	[-0.0393579 0.04217744]	1.374
3.25	tracking	[10.89845625 56.81944828]	[-0.70194746 0.92858943]	[-0.02440316 0.03589573]	1.131
3.50	tracking	[9.65550885 58.64010252]	[-0.83841389 1.30970887]	[-0.02901931 0.05056748]	1.302
3.75	tracking	[8.23235597 60.50698182]	[-1.23619838 1.30351224]	[-0.0535332 0.04931373]	2.130
4.00	tracking	[6.39567777 64.03791625]	[-1.15503226 1.74634035]	[-0.041127 0.06726881]	1.857
4.25	tracking	[4.44804033 67.59213356]	[-1.34549723 2.43164124]	[-0.04735042 0.09360748]	2.192
4.50	tracking	[2.51062878 71.0683005]	[-1.34586527 1.76381641]	[-0.05241547 0.06750527]	2.263
4.75	tracking	[0.52023422 74.8482838]	[-1.27444159 2.87469883]	[-0.04664574 0.11030782]	2.329
5.00	tracking	[-1.48405837 78.84717283]	[-1.50927743 1.76956613]	[-0.07032425 0.02893639]	2.603
5.25	not detected	[-1.8483176 79.13985795]	[5.40463525e-07 4.23737158e-08]	[2.91309800e-07 2.28394283e-08]	unavailable
5.50	not detected	[-1.84831806 79.13985791]	[8.30522341e-13 3.81666562e-14]	[2.05811279e-13 1.49861816e-14]	unavailable
5.75	not detected	[-1.84831806 79.13985791]	[0. 0.]	[2.78274672e-19 2.18174405e-20]	unavailable
6.00	not detected	[-1.84831806 79.13985791]	[0. 0.]	[1.96456363e-24 1.54026775e-25]	unavailable
6.25	not detected	[-1.84831806 79.13985791]	[0. 0.]	[1.38694270e-29 1.08739828e-30]	unavailable
6.50	not detected	[-1.84831806 79.13985791]	[0. 0.]	[9.79153862e-35 7.67681478e-36]	unavailable
6.75	not detected	[-1.84831806 79.13985791]	[0. 0.]	[6.91263080e-40 5.41967799e-41]	unavailable
7.00	not detected	[-1.84831806 79.13985791]	[0. 0.]	[4.88017935e-45 3.82618447e-46]	unavailable
7.25	tracking	[-8.01454081 101.8357679]	[-2.23055738 10.07915448]	[-0.08658674 0.38567393]	8.801
7.50	tracking	[-8.89112975 108.82861304]	[0.73005616 1.79108517]	[0.03417602 0.06858328]	2.138
7.75	tracking	[-7.89726833 110.54637205]	[0.61610506 0.06343894]	[0.02045746 0.00241799]	1.029
8.00	tracking	[-6.4903158 111.56707915]	[0.86176547 0.58738444]	[0.02486771 0.02246756]	1.564
8.25	tracking	[-4.90976594 112.08747441]	[0.93440644 0.28502778]	[0.03201568 0.01091427]	1.574
8.50	tracking	[-3.18493572 111.99781316]	[1.43932019 -0.29365267]	[0.0553076 -0.01120793]	2.332
8.75	tracking	[-1.33140196 111.60186622]	[1.63396637 -1.20146958]	[0.078549 -0.04591151]	2.739
9.00	tracking	[0.4592189 110.37666633]	[1.40514683 0.12133227]	[0.05366512 0.00466625]	2.347
9.25	tracking	[2.43054891 109.24732117]	[1.26407735 -1.3471666]	[0.04992584 -0.0654811]	2.230
9.50	tracking	[4.14580762 107.37280759]	[1.15473978 -1.33220574]	[0.03904079 -0.05101198]	2.065
9.75	tracking	[5.77344677 105.36712129]	[1.35284816 -1.86459245]	[0.05315115 -0.07128635]	2.510
10.00	tracking	[7.20951136 102.8795133]	[0.87443269 -1.88281001]	[0.0341504 -0.07065605]	1.913
10.25	tracking	[8.42457173 100.43687107]	[0.65781862 -2.09607775]	[0.0264168 -0.08015231]	1.834
10.50	tracking	[9.48192171 98.02473667]	[0.28763174 -2.13291196]	[0.00455155 -0.08184287]	1.744
10.75	tracking	[10.20496774 95.39659461]	[0.72828026 -1.83646175]	[0.04197134 -0.06964704]	1.656
11.00	tracking	[10.67676365 93.12614636]	[0.70849158 -1.80100795]	[0.03666594 -0.06845486]	1.590
11.25	tracking	[10.93986457 90.80598858]	[-0.17582514 -1.29291478]	[-0.00608751 -0.04944437]	1.317
11.50	tracking	[10.75901977 89.15459509]	[-0.18780579 -2.14499767]	[-0.01283832 -0.0824007]	2.073
11.75	tracking	[10.57719182 87.03852038]	[-0.24534374 -1.23463373]	[-0.00935577 -0.0472524]	1.386
Final joints: [9.96531035 85.32427186] degrees					
Final end-effector: [0.09388179 0.06709226] m					
Error to estimated target: 1.1973835470859828 mm					
Error to true target: 1.8082963011113569 mm					

1. Why was task error unavailable here but 88.384 mm in the unreachable test?
2. Which parts still operated during detection loss: vision, IK, or inner joint PD?
3. What fresh information allowed tracking to resume at 7.25s?
4. Is detection loss an information problem or a geometric reachability problem?

1. becu2 this time there is no target on sight
2. vision, inner PD ; IK is skipped
3. new successful detection
4. info problem.

→ final checkpoint

1. Put these in order: joint PD, pixel detection, IK, camera-to-robot transform, ray/plane intersection.
2. If IK error is zero but the arm misses the true cube by 1 mm, where did that remaining error originate?
3. Why must inner joint PD keep running faster than the camera loop?
4. What does the arm do in both unreachable and not detected, and what information differs between them?

1. pixel detection → ray/plane intersection → cam(→)robot. → IK → joint PD
2. from perception error
3. cuz arm's physical state changes cont. between cam. frames
4. on both it holds the current pos. ; task error info. differs between them.